NutriTrace

Projet Laravel · Esprit 5Twin1 · Équipe BrightMinds

Module Réclamations & Avis

Service après-vente intelligent : réclamations, avis clients, IA DeepSeek et agent conversationnel « Sahbi »

🤖 Agent IA à outils (function calling) · 🧠 6 fonctionnalités DeepSeek · ⏱️ Délais de traitement (SLA) · ⭐ Avis vérifiés & modérés · 🚨 Alertes qualité → rappel de lot · 🔐 3 rôles · Policies · Spatie · ✅ 50 tests automatisés

Branche Git : **ReclamationReponse** · Laravel 12 · PHP 8.2 · Novembre 2026

Document technique et fonctionnel : ce qui a été ajouté, pourquoi, et comment c'est implémenté.

Sommaire

1. Objectifs et réponse au cahier des charges
2. Point de départ : ce qui existait
3. Architecture du module
4. Modèle de données
5. Rôles et permissions
6. Réclamations : cycle de vie complet
7. Avis clients
8. Intelligence artificielle DeepSeek
9. Sahbi, l'agent après-vente
10. Alertes qualité et rappel de lot
11. Back office administrateur
12. Espace producteur
13. Tests et validation
14. Données de démonstration
15. Installation et scénario de démonstration
16. Fichiers créés et modifiés
17. Limites et pistes d'évolution

3	6	50	199
rôles dans le module (client, acteur, admin)	fonctionnalités IA + 1 agent à outils	routes du module	tests du projet au vert (dont 60 du module)

En une phrase : le module transforme une simple liste de réclamations en un véritable **service après-vente** : chaque réclamation est analysée par l'IA, suivie avec un délai garanti et une chronologie, les clients notent les produits, les problèmes récurrents sont détectés automatiquement jusqu'au rappel d'un lot, et un agent conversationnel, **Sahbi**, permet de tout faire en langage naturel (français, arabe, anglais, derja), avec une confirmation explicite avant toute écriture.

1. Objectifs et réponse au cahier des charges

Le cahier des charges demande, pour chaque module : une gestion d'au moins deux rôles, des fonctionnalités avancées et des fonctionnalités basées sur l'IA adaptées au module. Le tableau ci-dessous montre comment le module y répond.

Exigence	Réponse apportée
Gestion des rôles (au moins deux)	**Trois rôles** distincts dans le module : • **Consommateur** : dépose, suit, clôture et évalue ses réclamations ; publie, modifie et vote pour les avis. • **Acteur / producteur** : voit les réclamations et avis sur *ses* produits, répond aux clients, droit de réponse public sous les avis. • **Administrateur** : traite tout, copilote IA, analyse de responsabilité, modération, alertes qualité et rappel de lot. Implémentation : **Policies Laravel** (`ReclamationPolicy` , `AvisPolicy`) + **permissions Spatie** (`deposer-reclamations` , `publier-avis` , `repondre-avis` , `traiter-reclamations` , `moderer-avis`) sur les routes.
Fonctionnalités avancées	Référence lisible `REC-AAAA-NNNN` · délais de traitement (SLA) par priorité avec compte à rebours · chronologie de chaque dossier · commande et lot liés et vérifiés · clôture par le client et enquête de satisfaction · relance d'un dossier en retard · notifications (base + email) · export PDF du dossier · indicateurs (respect des délais, satisfaction, délai moyen) · avis avec badge « Achat vérifié », votes « utile », signalements, droit de réponse, tri et répartition des étoiles · commande planifiée.
Fonctionnalités IA	Six fonctionnalités **DeepSeek** : 1. Analyse d'une réclamation (catégorie, sentiment, urgence, langue, résumé, première réponse) ; 2. Copilote de réponse de l'administrateur (ton au choix, solution proposée, langue du client) ; 3. « Qui est responsable ? » : maillon de la chaîne en cause d'après la traçabilité du lot ; 4. Modération des avis et notes par aspect (goût, fraîcheur, emballage, prix, livraison) ; 5. Synthèse « Ce que les clients en pensent » sur la fiche produit ; 6. Alertes qualité : DeepSeek lit les signalements groupés et conseille ou non un rappel. Et un **agent conversationnel à outils**, Sahbi, qui agit dans l'application avec les droits de l'utilisateur.

Originalité par rapport aux autres modules

Les IA des autres modules aident surtout le producteur et le catalogue (fiche produit depuis une photo, recherche sémantique, météo, histoire de traçabilité, éco-score…). L'assistant du module Traçabilité est un **questions-réponses en lecture seule** sur un lot. Ce module couvre l'**après-vente**, la voix du client, et Sahbi est un **agent** : il **agit** (créer une réclamation, publier un avis, joindre un conseiller) après confirmation. Il **réutilise** les autres modules au lieu de les copier : il lit les commandes du module Commandes, croise les étapes de traçabilité pour désigner un responsable et déclenche le statut « rappelé » d'un lot du module Traçabilité.

2. Point de départ : ce qui existait

Sur la branche `main`, une première version CRUD du module existait déjà (commit « crud reclma ») :

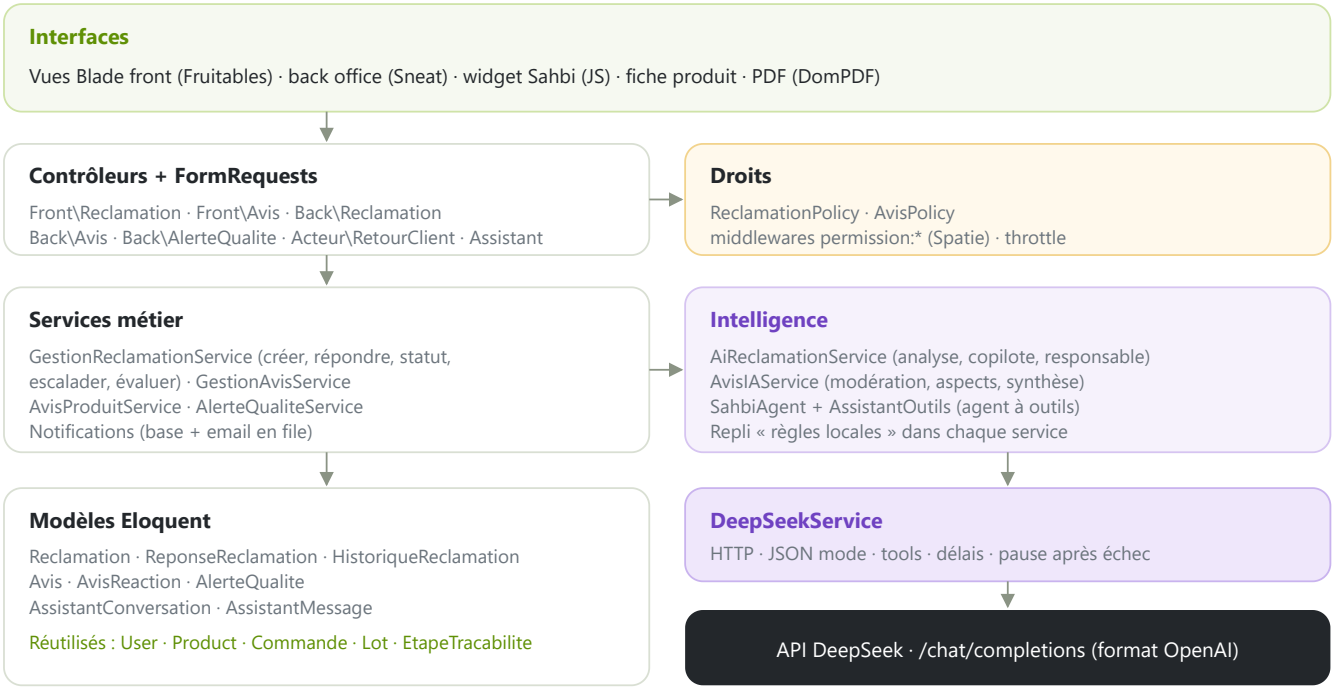
- tables `reclamations` et `reponse_reclamations`, modèles, contrôleurs front et back, 10 tests ;
- une « IA » par **mots-clés** (aucun appel à un vrai modèle) ;
- numéro de commande libre (ni clé étrangère ni contrôle d'appartenance) ;
- permissions Spatie `deposer-reclamations` / `traiter-reclamations` définies mais **jamais utilisées** ;
- la partie **Avis** n'existait pas : l'onglet « Avis » de la page produit était un gabarit HTML statique.

Ce que la branche ReclamationReponse ajoute

Domaine	Ajout
Réclamations	Référence, SLA, chronologie, catégories IA, lot/commande vérifiés, clôture, satisfaction, relance, PDF, notifications, indicateurs.
Avis (nouveau)	Tables `avis` et `avis_reactions`, publication, achat vérifié, modération IA, votes, signalements, droit de réponse, conversion en réclamation, synthèse IA.
IA	`DeepSeekService` (client API robuste) et 6 fonctionnalités, toutes avec repli local sans clé.
Agent Sahbi (nouveau)	Widget sur toutes les pages, 18 outils répartis par rôle, cartes de confirmation, accueil proactif, mode hors ligne.
Alertes qualité (nouveau)	Détection des problèmes récurrents, analyse IA, rappel de lot, information des clients, commande planifiée.
Rôles	2 Policies, 3 nouvelles permissions, routes protégées par permission.
Qualité	50 nouveaux tests (60 au total pour le module), seeder de démonstration complet.

3. Architecture du module

Le module suit une architecture en couches. **Les contrôleurs restent minces** : ils valident, vérifient les droits puis délèguent à des services métier. Ces services sont **partagés** par les formulaires, l'agent Sahbi et la conversion d'avis : une réclamation créée par Sahbi suit exactement les mêmes règles qu'une réclamation saisie au formulaire.



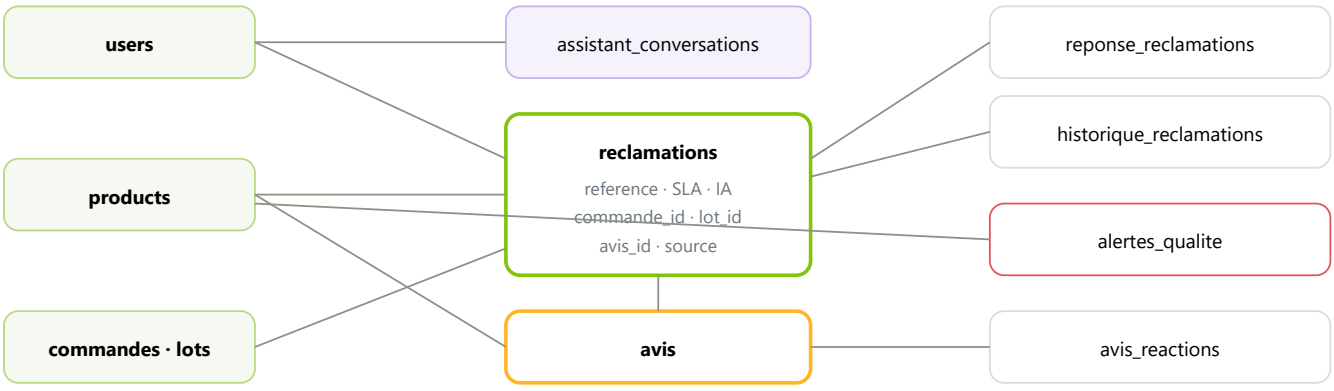
Principes de conception

- **Un seul chemin d'écriture** : `GestionReclamationService::creer()` est appelé par le formulaire client, le back office, Sahbi et la conversion d'avis. Analyse IA, chronologie et notifications sont donc garanties partout.
- **L'IA propose, l'humain décide** : l'IA ne change jamais un statut, ne publie rien et ne rappelle aucun lot seule. Le copilote remplit une zone de texte que l'administrateur relit, Sahbi affiche une carte à confirmer et l'alerte qualité attend le clic de l'administrateur.
- **Toujours fonctionnel** : sans clé, sans réseau ou en cas de quota dépassé, chaque fonction IA bascule sur des règles locales. Le module reste complet en démonstration hors ligne et dans les tests.
- **Ne jamais faire confiance au modèle** : chaque valeur renvoyée par DeepSeek est comparée à une liste fermée (catégories, priorités, langues…) ; une valeur inventée est remplacée par la valeur locale.

4. Modèle de données

Quatre migrations ajoutées (préfixe `2026_11_03_1000xx`), sans modifier les tables des autres modules.

Table	Rôle et colonnes principales
`reclamations` **enrichie**	`reference` (unique, REC-AAAA-NNNN) · `lot_id` → lots · `avis_id` → avis · `source` (formulaire / assistant / avis) · `categorie`, `langue`, `resume_ia` · `responsable_suggere`, `responsable_explication` · `echeance_at`, `resolue_at` (SLA) · `satisfaction_note`, `satisfaction_commentaire`
`historique_reclamations` **nouvelle**	Chronologie : `type` (creation, statut, reponse, ia, responsable, escalade, satisfaction, rappel), `libelle`, auteur, date.
`avis` **nouvelle**	`note` 1-5, `titre`, `commentaire`, `statut` (publie / en_moderation / masque), `achat_verifie`, `sentiment`, `aspects` (JSON), `moderation_raison`, `probleme_detecte`, `reponse_producteur`, compteurs. **Contrainte unique (user_id, produit_id)** : un avis par client et par produit.
`avis_reactions` **nouvelle**	Vote « utile » ou signalement (avec motif). Unique (avis, user, type).
`alertes_qualite` **nouvelle**	Produit, lot le plus cité, `signaux` (ids JSON), `signature` (détecte les changements), titre, résumé, gravité, recommandation, `rappel_conseille`, `motif_rappel`, `analyse_ia`, statut (ouverte / rappel_declenche / classee), traitée par/le.
`assistant_conversations` `assistant_messages` **nouvelles**	Conversation Sahbi par utilisateur ; messages avec `action` (carte : outil, arguments revalidés, statut) et `liens` (boutons).
`notifications`	Table standard Laravel (créée si absente).



Les relations réutilisent les tables des autres modules (`users`, `products`, `commandes`, `lots`) via des clés étrangères, sans les modifier.

5. Rôles et permissions

Le projet utilise **spatie/laravel-permission**. Chaque compte reçoit automatiquement un rôle Spatie aligné sur ses colonnes `role` / `type_acteur`. Le module ajoute trois permissions et utilise enfin les deux qui existaient déjà.

Permission	Consommateur	Producteur	Transformateur	Admin	Utilisée pour
`deposer-reclamations`	✓		✓	✓	Créer une réclamation (Policy `create`)
`publier-avis` nouv.	✓		✓	—*	Donner un avis
`repondre-avis` nouv.		✓	✓	✓	Droit de réponse sous les avis de ses produits
`traiter-reclamations`				✓	Toutes les routes back office réclamations + alertes
`moderer-avis` nouv.				✓	File de modération des avis

* L'administrateur reçoit toutes les permissions, mais la Policy l'empêche de noter un produit : il modère, il ne note pas. Le transformateur achète des matières premières : il peut donc aussi réclamer et noter.

Règles fines dans les Policies

ReclamationPolicy

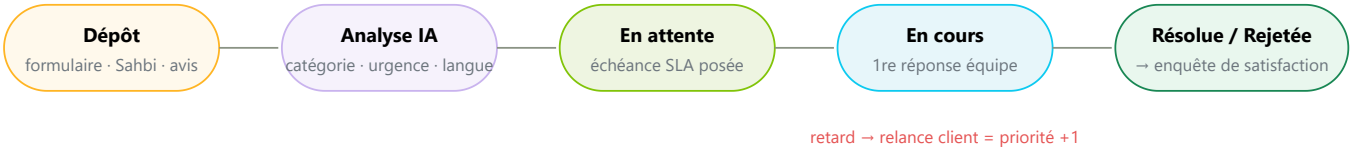
- **view** : auteur, administrateur, ou acteur *vendeur du produit* ;
- **update / delete** : auteur, tant que le dossier est « en attente » ;
- **repondre** : participants du dossier, tant qu'il n'est pas clôturé ;
- **cloturer** : l'auteur indique que son problème est réglé ;
- **evaluer** : l'auteur, une seule fois, après clôture ;
- **traiter** : administrateur avec `traiter-reclamations`.

AvisPolicy

- **create** : permission `publier-avis`, produit publié, pas son propre produit, pas d'avis existant ;
- **update** : l'auteur ; **delete** : l'auteur ou un modérateur ;
- **reagir** (utile / signaler) : pas sur son propre avis ;
- **repondre** : l'acteur qui vend le produit ;
- **convertir** : l'auteur d'un avis ≤ 2★ ;
- **moderer** : administrateur avec `moderer-avis`.

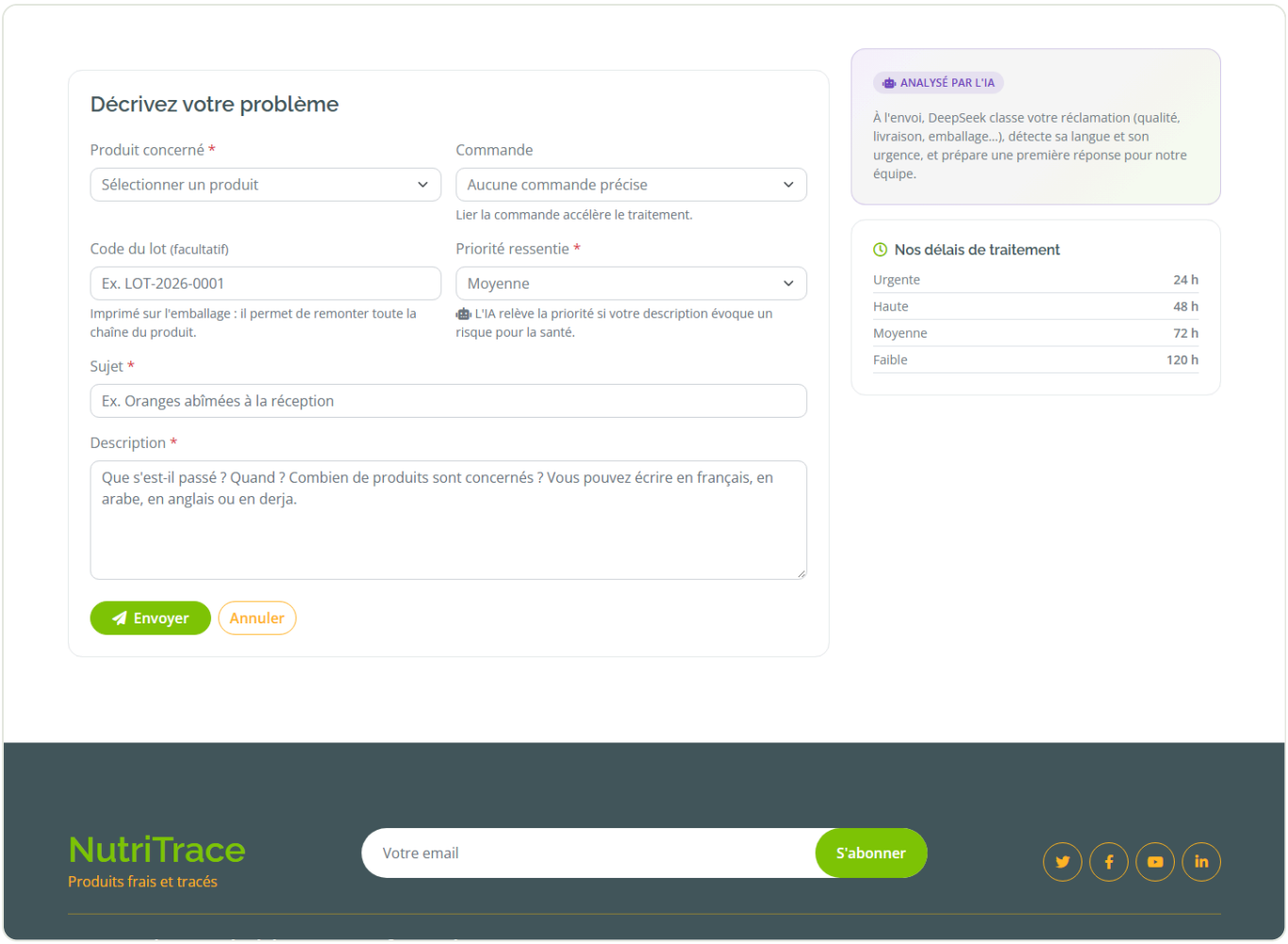
> **Même logique pour l'agent :** les outils de Sahbi sont filtrés par rôle (un client n'a pas l'outil `rechercher_reclamations` de l'admin), et chaque outil interroge la base avec l'utilisateur connecté. Même si on manipule le modèle par un prompt, il ne peut voir ni modifier les données d'un autre client : les tests le vérifient.

6. Réclamations : cycle de vie complet

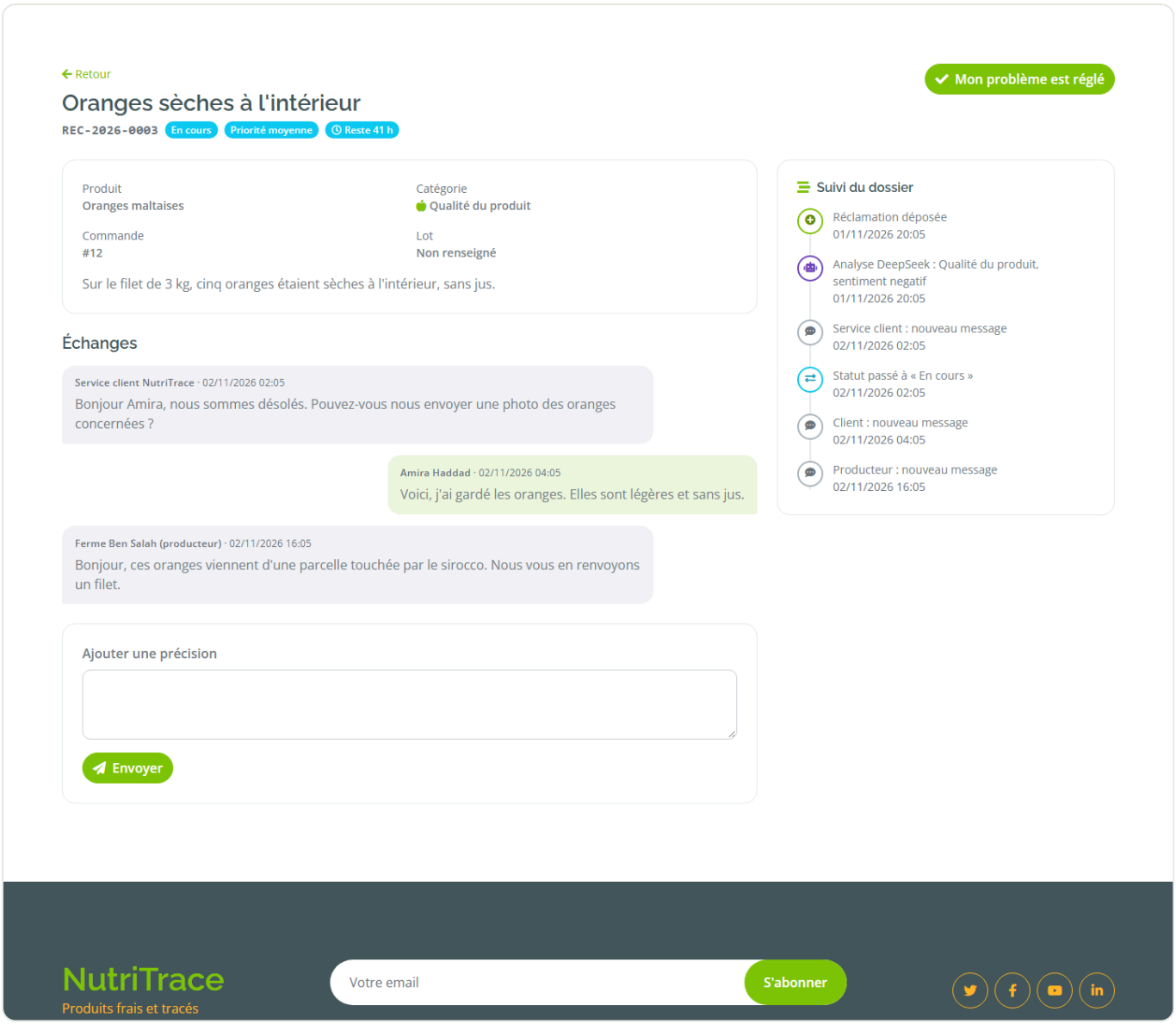


Fonctionnalités et implémentation

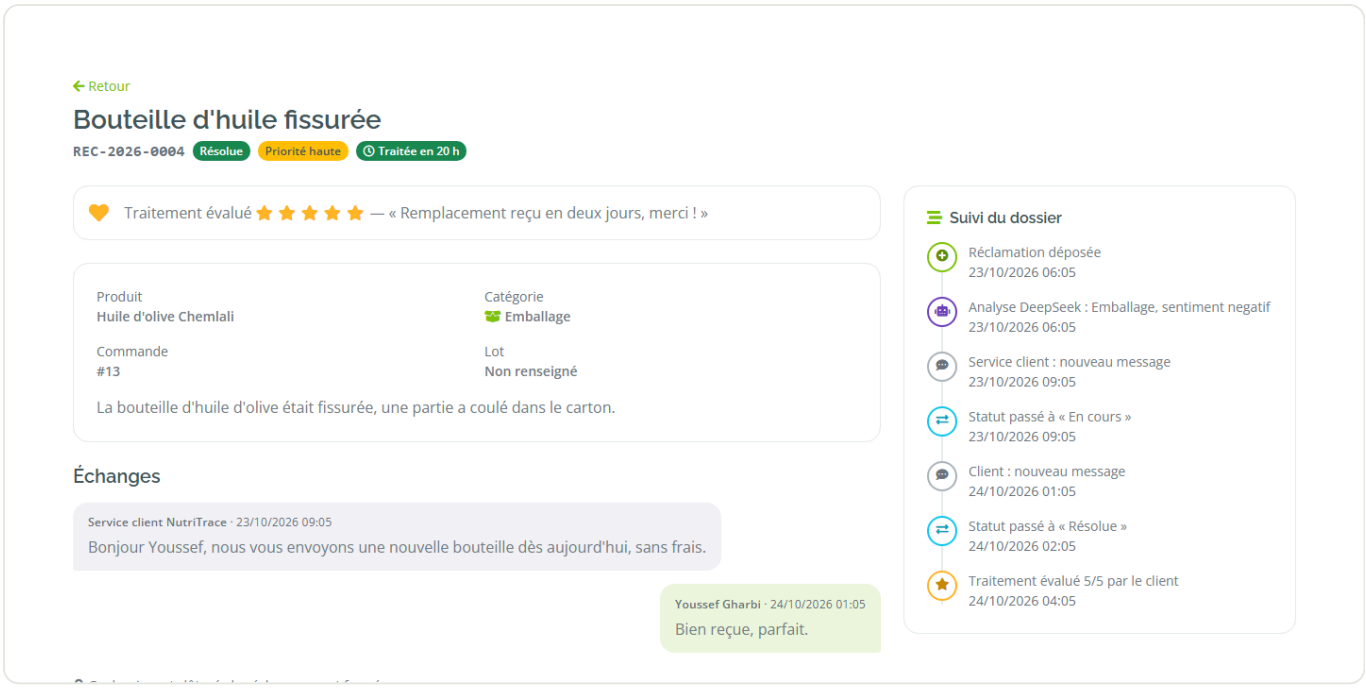
Fonctionnalité	Implémentation
Référence `REC-2026-0042`	Événement `created` du modèle : la référence reprend l'identifiant et l'année, unique en base.
Délais de traitement (SLA)	`config/reclamations.php` : urgente 24 h, haute 48 h, moyenne 72 h, faible 120 h. `echeance_at` est calculée à la création et recalculée si la priorité change. Méthodes `estEnRetard()`, `slaLibelle()` (« Reste 5 h », « En retard de 2 j », « Traitée en 20 h »), `slaBadge()` (bleu, orange quand il reste moins d'un quart du délai, rouge en retard), `slaRespecte()`.
Priorité relevée par l'IA	Le client garde sa priorité, sauf si l'IA détecte plus grave (risque sanitaire, remboursement) : la priorité monte et la chronologie l'indique.
Commande et lot vérifiés	La liste des commandes du client filtre les produits (JS). Validation `Rule::exists('commandes')->where('user_id', …)`. Le code du lot doit correspondre au produit choisi.
Chronologie	Table `historique_reclamations`, méthode `journaliser()` appelée par le service à chaque événement ; affichage en frise côté client et côté admin.
Échanges à trois	Client, service client et producteur vendeur ; bulles différenciées ; réponses fermées après clôture.
Clôture et satisfaction	Bouton « Mon problème est réglé », puis enquête 1 à 5 étoiles et commentaire, une seule fois. Moyenne affichée dans les indicateurs admin.
Relance	Dossier en retard : bouton « Relancer le service client », la priorité monte d'un cran et les administrateurs sont notifiés.
Notifications	`ReclamationNotification` (nouvelle, réponse, statut, escalade) et `RappelProduitNotification`. Canal base de données immédiat (`viaConnections` → sync), email en file d'attente pour ne jamais ralentir la page. Badge de non-lus dans la barre de navigation.
Dossier PDF	DomPDF : réclamation, analyse IA, échanges, chronologie.



Formulaire : commande et lot liés, priorité ressentie, délais affichés, analyse IA annoncée.



Dossier côté client : référence, statut, priorité, délai restant, échanges avec le service client et le producteur, chronologie.

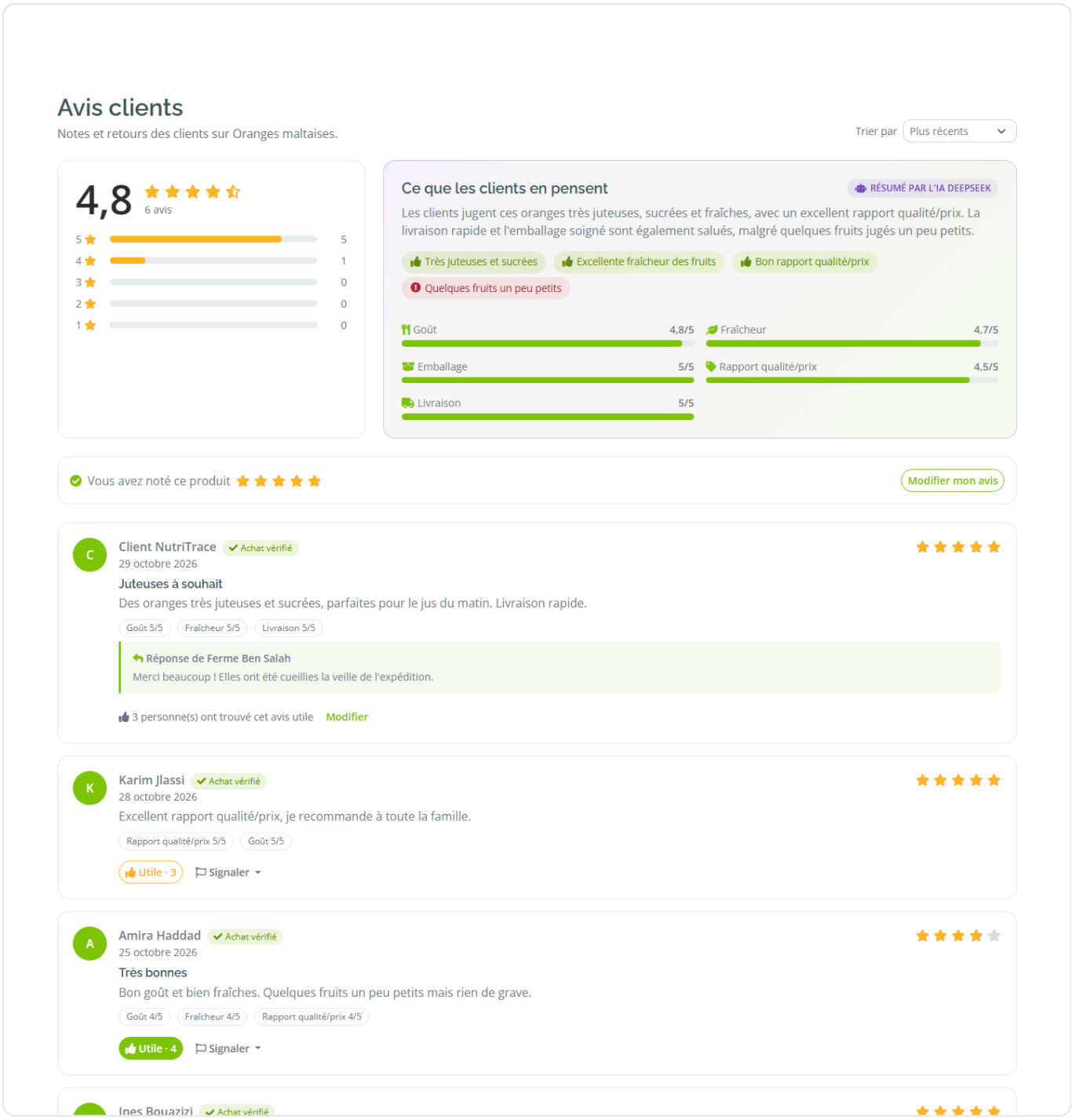


Dossier résolu « en 20 h » (délai respecté) et évalué 5/5 par le client ; les échanges sont fermés.

7. Avis clients

Les avis remplacent le gabarit statique de la page produit par une vraie fonctionnalité, directement sur la fiche produit du catalogue.

Fonctionnalité	Implémentation
Publication	Note 1-5 avec sélecteur d'étoiles accessible (radios, clavier), titre facultatif, commentaire (10 à 2000 caractères). Un seul avis par produit (contrainte unique + Policy).
Achat vérifié	`Avis::estAchatVerifie()` : le client a une commande **livrée** contenant ce produit, ce qui affiche le badge « Achat vérifié ».
Modération IA	À la publication, `AvisIAService::analyser()` renvoie une décision publier / modérer. Un avis injurieux, publicitaire ou contenant un téléphone part « en modération » et n'est pas visible. Une critique négative mais polie est publiée.
Notes par aspect	L'IA extrait goût, fraîcheur, emballage, prix, livraison (1-5) du texte libre ; affichés en puces sous l'avis et agrégés par produit.
Synthèse IA	Bloc « Ce que les clients en pensent » : résumé, points forts et points faibles, barres par aspect. Mis en cache et recalculé seulement quand les avis changent.
Répartition, tri	Moyenne, histogramme 5→1 étoiles, tri par plus récents, plus utiles, meilleures notes ou plus critiques, pagination.
Votes et signalements	« Utile » (bascule) ; signalement avec motif. À 3 signalements, l'avis repasse en modération automatiquement.
Droit de réponse	Le producteur répond publiquement sous l'avis (encadré « Réponse de … »).
Avis → réclamation	Si un avis ≤ 2★ décrit un vrai problème (`probleme_detecte`), un bandeau propose d'ouvrir une réclamation **en un clic** : sujet, texte et commande livrée sont repris, et la réclamation est liée à l'avis (`source = avis`). L'opération est idempotente.



Fiche produit : moyenne et répartition, synthèse rédigée par DeepSeek, notes par aspect, avis vérifiés, réponse du producteur, votes « utile ».

8. Intelligence artificielle DeepSeek

8.1 Le client d'API : DeepSeekService

DeepSeek expose une API au format OpenAI (`POST /chat/completions`). Le service centralise tous les appels :

- **Configuration** : `config/services.php` → clé, modèle (`deepseek-chat`), URL, délai, lus depuis `.env` (`DEEPSEEK_API_KEY` …). La clé n'est jamais versionnée.
- **Trois méthodes** : `json()` (mode `response_format: json_object` avec schéma décrit dans le prompt), `texte()` , `discuter()` (messages + **tools** pour l'agent).
- **Robustesse** : délai de connexion de 8 s et délai total de 30 s ; toute erreur est journalisée et la méthode renvoie `null` . Après un échec, une **pause** en cache (2 min, 15 min si clé refusée ou quota) évite de faire attendre chaque page.
- **Lecture tolérante** : `lireJson()` récupère l'objet JSON même entouré de balises ```` ```json ````.
- **Tests** : `phpunit.xml` force une clé vide ; les tests IA simulent l'API avec `Http::fake` , et aucun appel réseau réel n'est possible (`preventStrayRequests`).

8.2 Les six fonctionnalités

Fonction	Ce que fait l'IA	Garde-fous / repli
Analyse de réclamation AiReclamationService::analyserReclamation	Sentiment, priorité, catégorie (7), langue (fr / ar / en / derja), résumé d'une phrase, premier message de réponse **dans la langue du client**.	Valeurs hors liste remplacées ; « urgente » réservée aux risques pour la santé ; repli par mots-clés (y compris détection de l'arabe).
Copilote de réponse redigerReponse	Brouillon complet selon le ton (empathique, formel, bref), qui tient compte des échanges précédents et du montant de la commande, propose une solution (remplacement, remboursement, bon…) et répond dans la langue du client.	Ne promet ni délai ni montant ; l'admin relit avant l'envoi ; modèles locaux par catégorie et par ton.
Qui est responsable ? analyserResponsabilite	Croise la réclamation avec les **étapes de traçabilité du lot** (dates, lieux, températures, acteurs) et les anomalies déjà calculées par le module Traçabilité, puis désigne le maillon probable avec un indice de confiance.	N'invente aucune étape ; confiance plus faible sans lot ; repli : chaîne du froid, sinon règles selon la catégorie.
Modération d'avis AvisIAService::analyser	Publier ou modérer, raison, sentiment, notes par aspect, détection d'un problème concret.	Les règles locales (insultes, liens, téléphones) gardent le dernier mot.
Synthèse produit AvisIAService::synthese	Résumé neutre des 40 derniers avis, 3 points forts, 3 points faibles.	Aucun nom de client ; cache d'une semaine indexé sur le nombre et la date des avis ; repli statistique.

Alerte qualité AlerteQualiteService::analyser	Dit si les signalements décrivent le même problème, titre, gravité, recommandation, rappel conseillé et **texte du motif de rappel** destiné aux consommateurs.	Le rappel reste une décision de l'administrateur ; résumé local si l'IA est indisponible.

8.3 Exemples réels obtenus avec DeepSeek pendant les tests

Réclamation en derja : « Klit el fraise elli chrithom el bareh w weldi mridh, fama 3fen abyedh fihom »


```
{ "sentiment": "negatif", "priorite": "urgente", "categorie": "securite_alimentaire", "langue": "tn",
  "resume": "Client signale moisissure blanche sur fraises du Cap Bon et maladie de son enfant après
consommation.",
  "suggestion": "Salam, nchokrek barcha 3al tbaligh w n7ess bik barcha. N7eb na3ref 3la 7al weldek
9bal kol chay…",
  "source": "ia" }
```

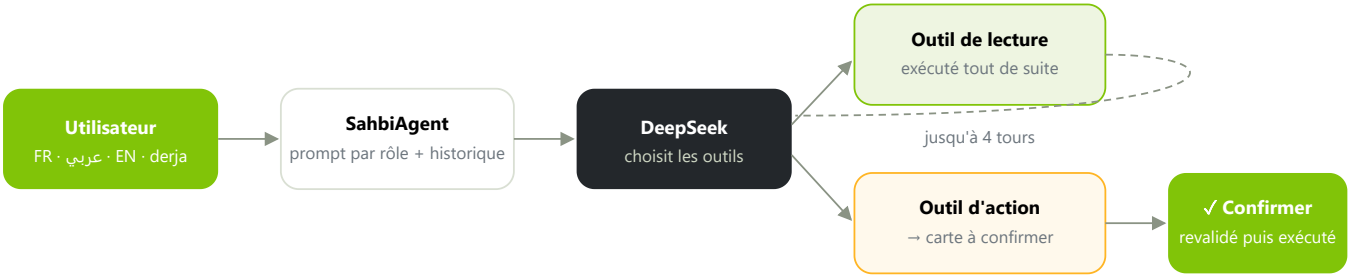

La langue est reconnue, l'urgence aussi (un enfant malade), et la première réponse est rédigée en derja.

Copilote, ton « bref », réclamation d'allergie : « Bonjour Nour, nous sommes vraiment désolés d'apprendre la réaction de votre fille. Par précaution, nous vous demandons de ne plus consommer le produit et de conserver l'emballage. Pouvez-vous nous envoyer une photo de l'emballage et le numéro de lot ? … » **Solution : demande d'informations complémentaires** — 2,2 s

Modération : « L'huile est bonne mais la bouteille est arrivée fissurée… » → **publié**, aspects `{goût: 4, emballage: 1, livraison: 1}`, problème détecté. « Vendeur voleur, contactez-moi sur insta @promo_huile » → **modéré** : « Publicité et lien vers un contact externe ».

9. Sahbi, l'agent après-vente

Sahbi (« mon ami » en tunisien) est un widget flottant présent sur toutes les pages, en front comme en back office. Ce n'est pas un simple chatbot de questions-réponses : c'est un **agent à outils**. DeepSeek décide quels outils appeler, Laravel les exécute avec les droits de l'utilisateur, et le modèle formule la réponse.



Outils disponibles selon le rôle

Rôle	Lecture	Actions (avec confirmation)
Client	`mes_commandes`, `mes_reclamations`, `details_reclamation`, `rechercher_produit`, `produits_a_evaluer`	`creer_reclamation`, `ajouter_message`, `publier_avis`, `escalader` (conseiller humain)
Acteur	outils client + `reclamations_mes_produits`, `avis_mes_produits`, `synthese_qualite`	idem client
Administrateur	`reclamations_prioritaires`, `statistiques`, `rechercher_reclamations`, `details_reclamation`, `alertes_qualite`, `avis_a_moderer`, `rediger_reponse`	—

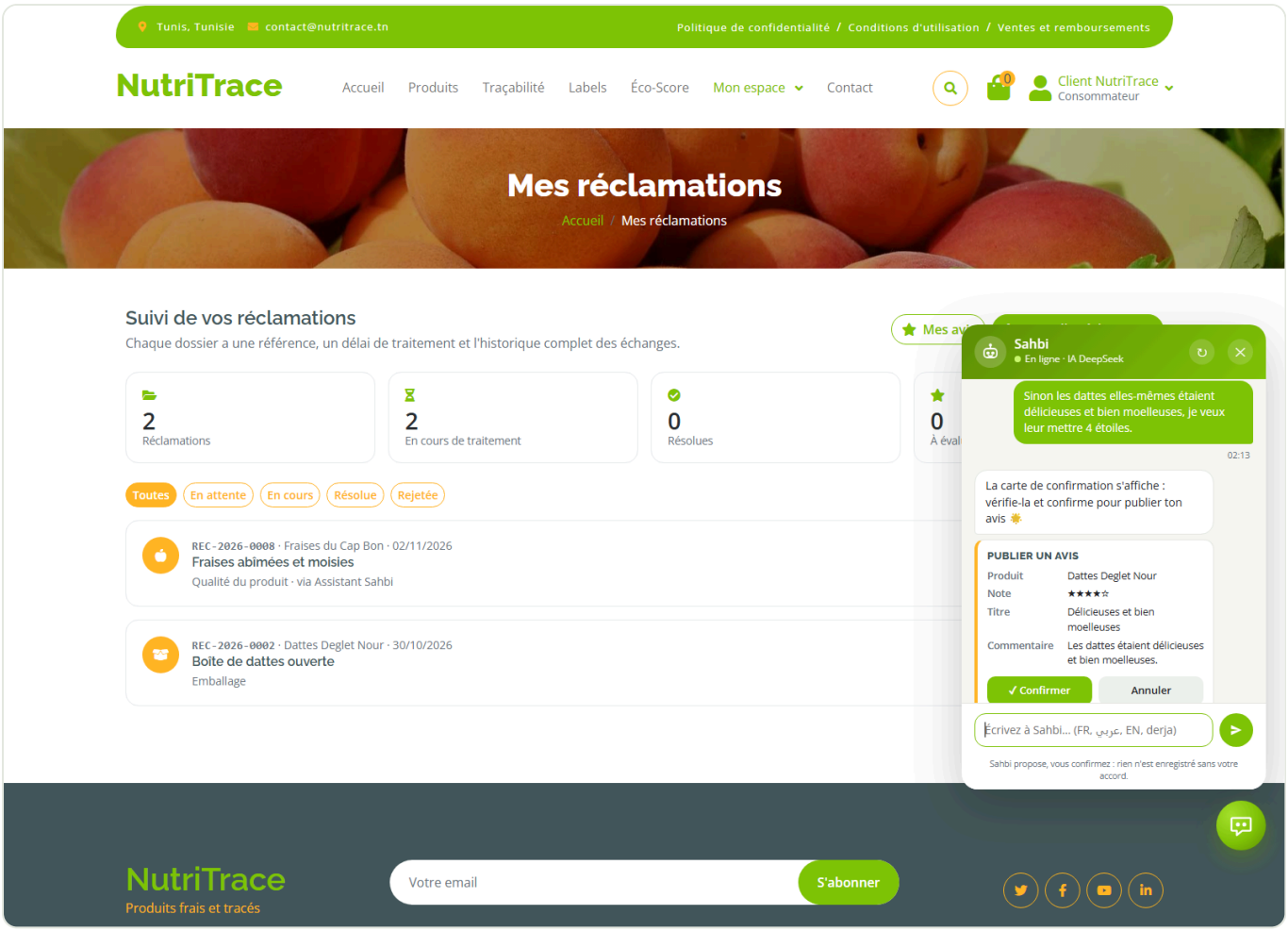
Ce qui le rend sûr et utile

🔐 Sécurité

- Les outils proposés dépendent du rôle ; un outil non autorisé est refusé côté serveur.
- Les arguments du modèle sont **revalidés** : commande appartenant au client, lot du bon produit, avis unique…
- Aucune écriture sans clic sur « Confirmer » ; un autre utilisateur ne peut pas confirmer la carte (403).
- Texte affiché avec `textContent` (pas d'injection HTML), liens limités au domaine, 20 messages par minute.

✨ Expérience

- **Accueil proactif** : « Vous avez reçu Fraises de Nabeul : comment les avez-vous trouvées ? », avec une bulle d'accroche et une pastille.
- Suggestions rapides adaptées au rôle, boutons vers les pages utiles.
- Escalade vers un humain avec la transcription jointe au dossier.
- **Mode hors ligne** : sans IA, réponses par mots-clés à partir des mêmes outils.



Le client écrit librement ; Sahbi prépare l'avis à partir de ses mots et affiche une carte « Publier un avis » à confirmer (conversation réelle avec DeepSeek).

Conversation réelle (client, extraits)

Salam, chnowa sar fi réclamation mte3i mta3 el harissa ?

Ta réclamation harissa (REC-2026-0012) est **en cours** : l'équipe t'a répondu qu'un nouveau pot est en route. Il reste 11 h avant la prochaine échéance. 😊

Les tomates de ma dernière commande étaient pourries, 3 sur 5. Je veux faire une réclamation.

Ta dernière commande livrée avec des tomates est la n°11 du 26/10. Tu as un code lot imprimé sur l'emballage ? Sinon je dépose la réclamation sans.

Non, je n'ai pas le code du lot. Vas-y.

J'ai préparé la réclamation : vérifie la carte et confirme.
Carte : Tomates fraîches · commande #11 · « 3 tomates sur 5 pourries »

✅ C'est fait : votre réclamation **REC-2026-0017** est enregistrée (priorité haute, relevée par l'IA ; délai de traitement : reste 47 h).

Conversation réelle (administrateur)

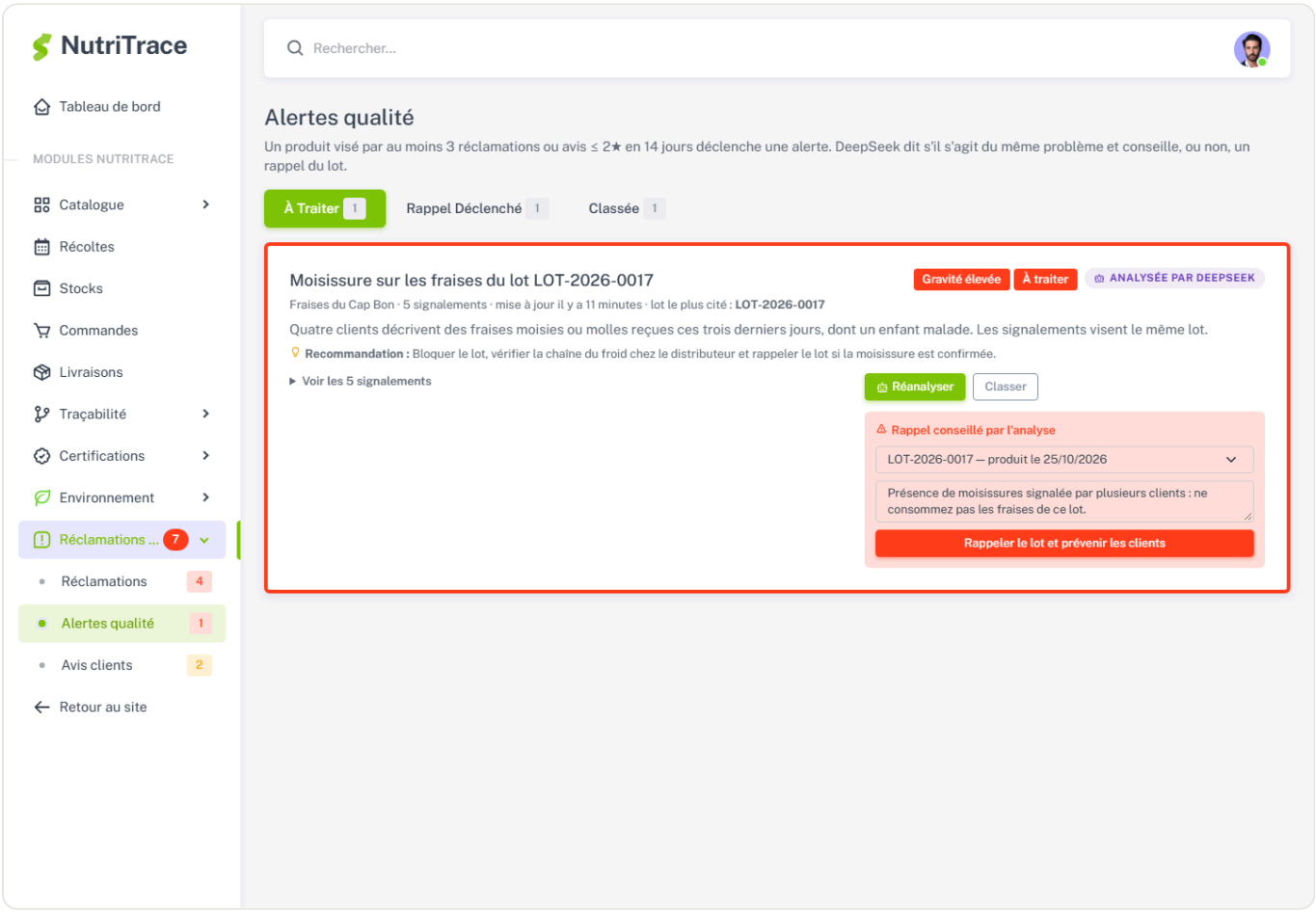
Quels dossiers dois-je traiter en priorité, et y a-t-il une alerte qualité ?

D'abord les 3 réclamations « Piments du Cap Bon » en retard (REC-2026-0014 et 0015 urgentes, 4 j et 3 j de retard, puis 0016), ensuite REC-2026-0002 (dattes, 24 h de retard). Oui, une alerte qualité élevée est ouverte sur les **Fraises du Cap Bon**, lot LOT-2026-0017 : 5 signalements de moisissure, dont un enfant malade, avec rappel conseillé. Je vous rédige les brouillons de réponse ?

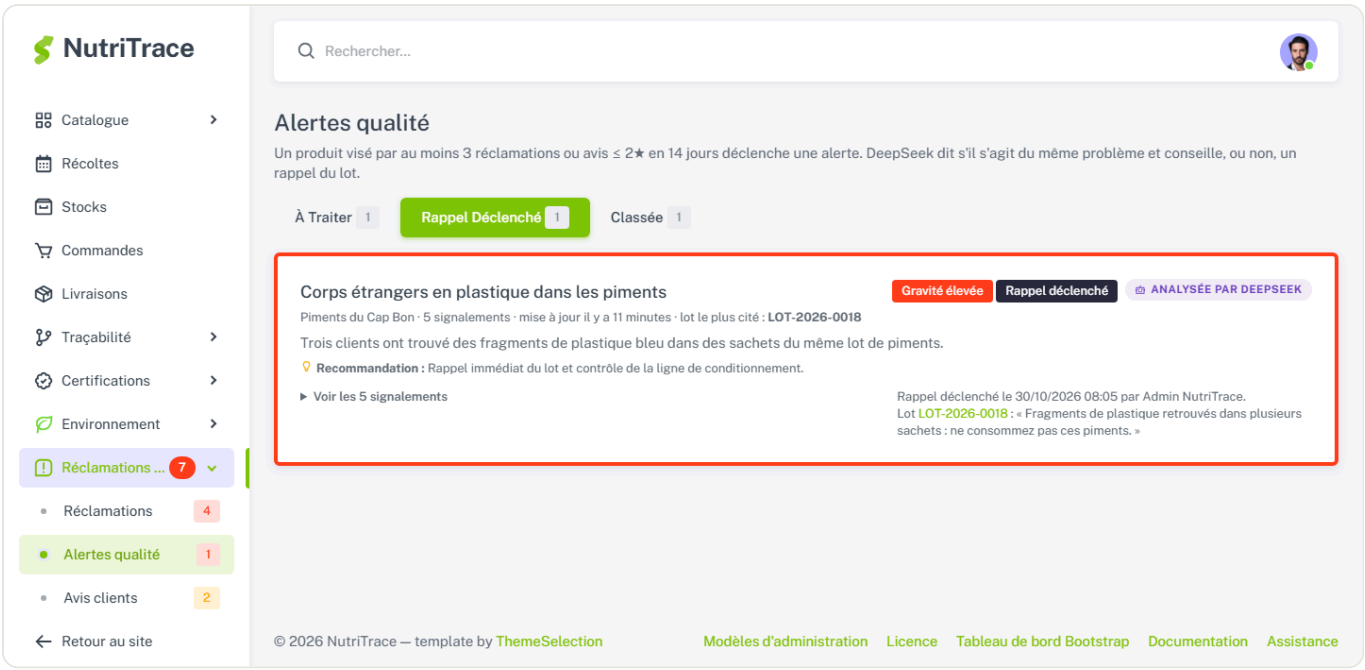
10. Alertes qualité et rappel de lot

Quand plusieurs clients se plaignent du même produit en peu de temps, il s'agit peut-être d'un incident sur un lot. Le module le détecte et relie l'après-vente à la traçabilité.

1. **Détection locale** (`AlerteQualiteService::detecter` , sans IA) : au moins **3 signaux** en **14 jours** sur un même produit (configurable). Les signaux sont les réclamations de qualité, sécurité, emballage ou quantité, plus les avis ≤ 2★. Un retard de livraison ou une question n'en font pas partie. Le lot le plus cité est retenu. Une *signature* des signaux évite les doublons et indique si l'analyse IA est périmée.
2. **Analyse DeepSeek** : même problème ou non, gravité, recommandation, rappel conseillé, motif rédigé pour les consommateurs.
3. **Décision de l'administrateur** : « Rappeler le lot et prévenir les clients » ou « Classer ».
4. **Rappel** : le lot passe au statut `rappele` avec son motif (module Traçabilité, visible sur la page publique du lot et via QR code). Chaque réclamation liée reçoit un événement « rappel ». Les auteurs des signalements et les acheteurs des 60 derniers jours reçoivent une notification (base + email).
5. **Automatisation** : `php artisan reclamations:alertes --ia` , planifiée toutes les heures (`routes/console.php`).



Alerte ouverte analysée par DeepSeek : gravité élevée, recommandation, motif de rappel prérempli, choix du lot.



Alerte traitée : le lot LOT-2026-0018 a été rappelé, avec la date, l'administrateur et le motif affiché aux consommateurs.

11. Back office administrateur

- **Indicateurs** : dossiers ouverts, en retard, urgents, taux de délais respectés (90 jours), satisfaction moyenne, délai moyen de traitement.
- **Tri intelligent** : dossiers en retard, puis urgents, puis récents ; lignes en retard surlignées ; filtres par statut, priorité, catégorie, « en retard », recherche (référence, client, produit).
- **Dossier** : panneau d'analyse IA, « Qui est responsable ? », copilote (ton, génération, solution proposée, langue), suggestion initiale, chronologie, statut, autres réclamations sur le même produit, export PDF.
- **Menu** « Réclamations & Avis » avec compteurs : dossiers en retard, alertes ouvertes, avis à modérer.

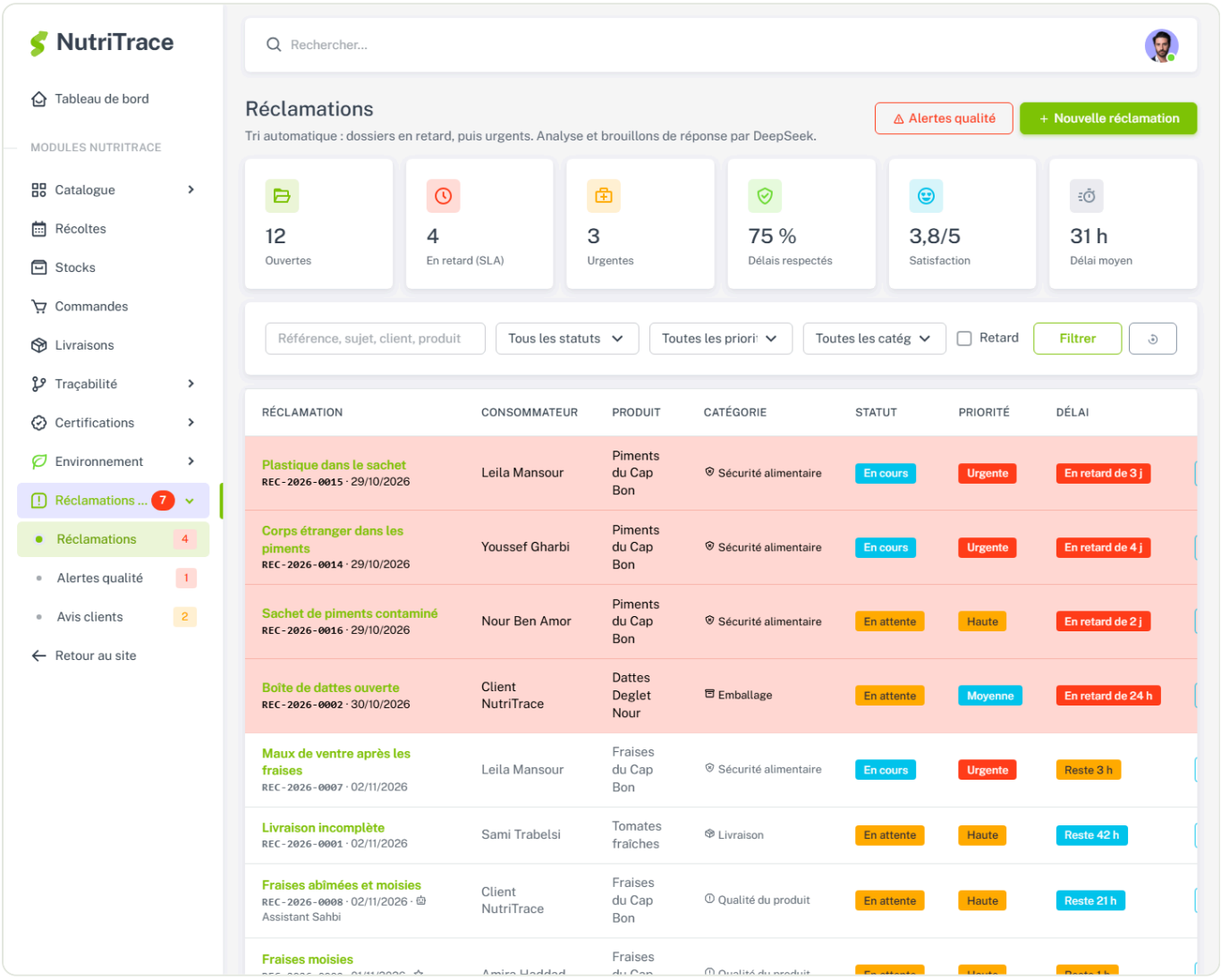
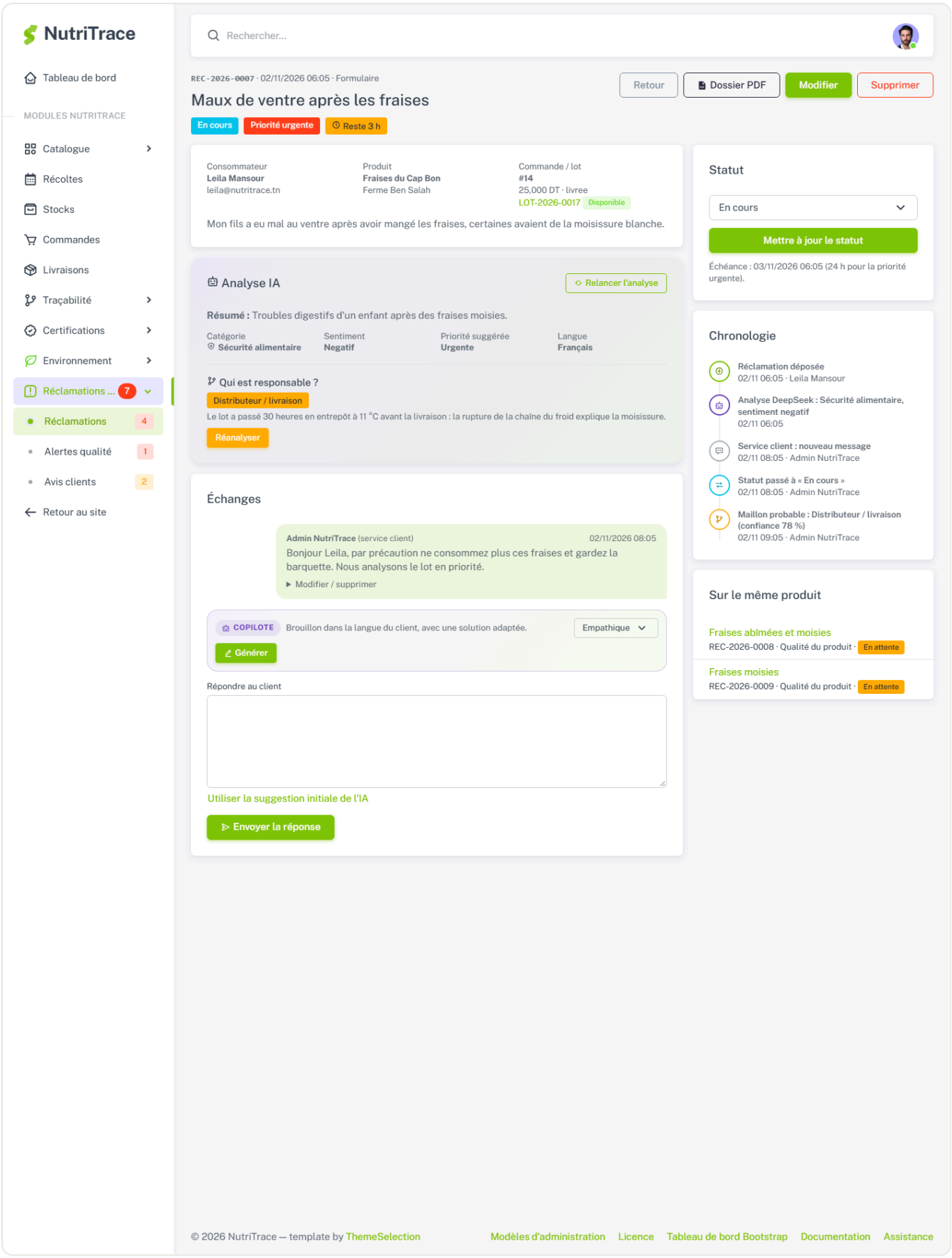
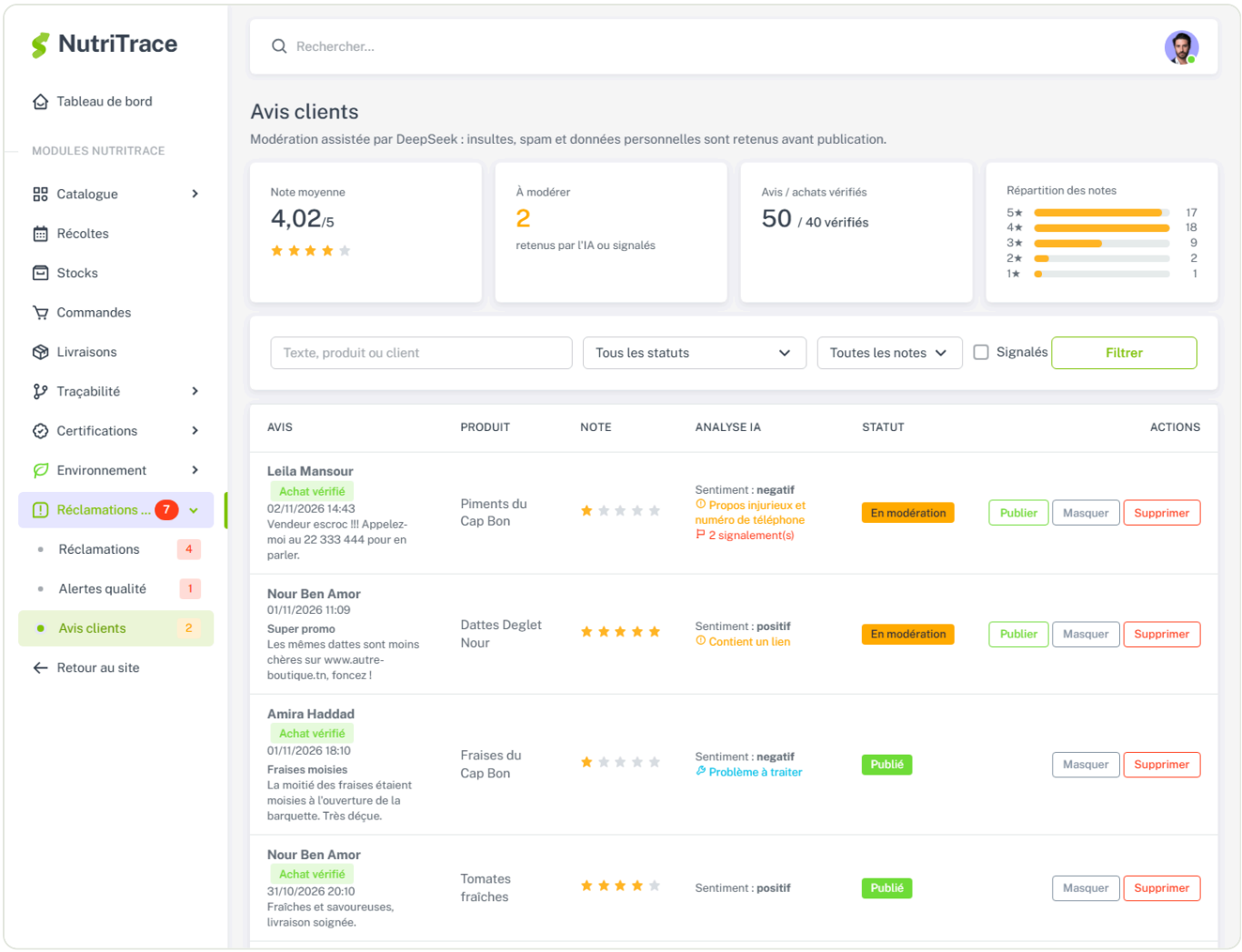


Tableau des réclamations : indicateurs, filtres, tri par urgence, délais en rouge.



Dossier admin : analyse IA, maillon responsable (la rupture de la chaîne du froid chez le distributeur explique la moisissure), échanges, copilote, chronologie.

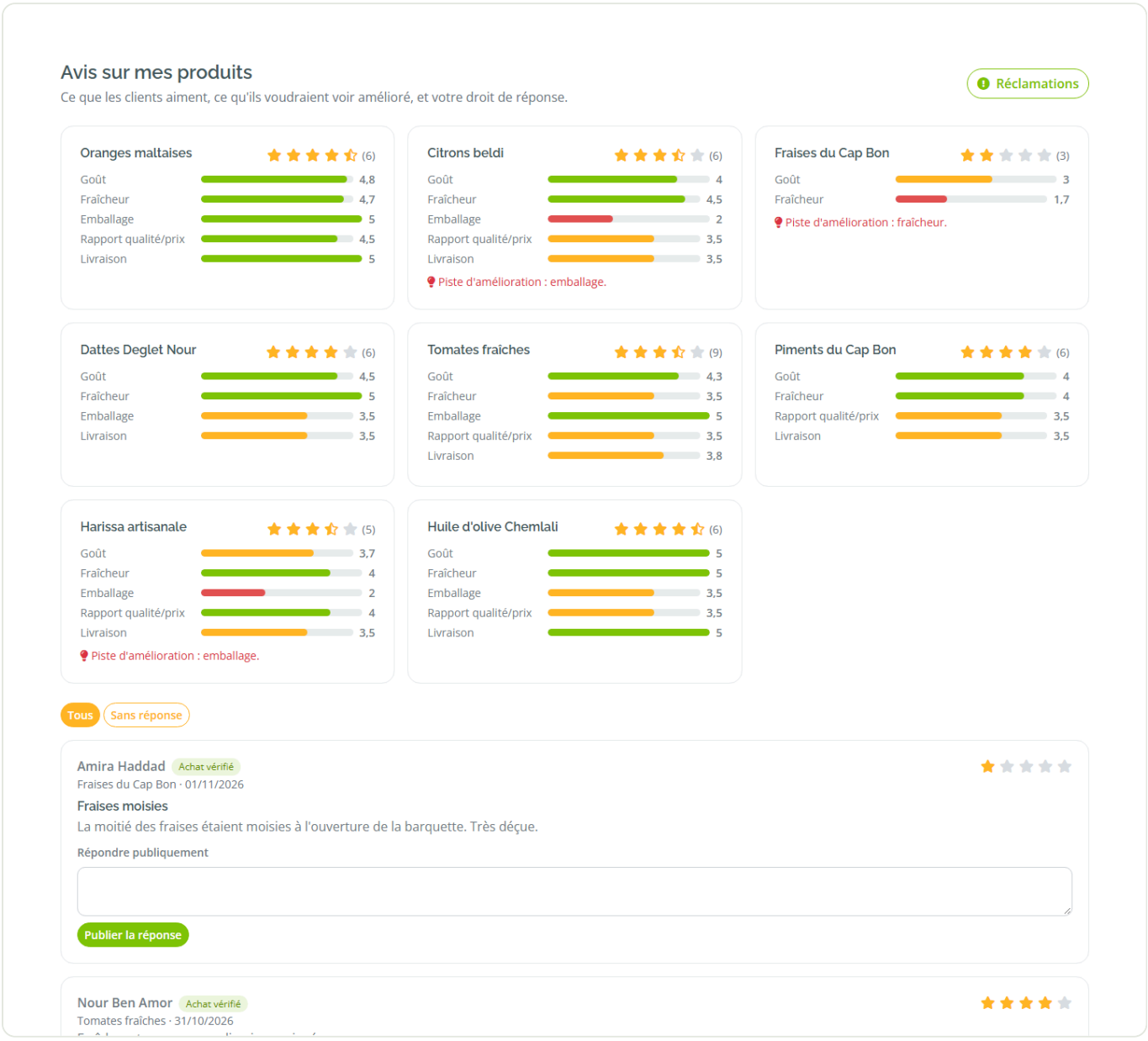


Modération des avis : raisons de l'IA, signalements, achats vérifiés, répartition des notes.

12. Espace producteur

L'acteur dispose de deux pages, « Réclamations sur mes produits » et « Avis sur mes produits » :

- une carte par produit (note moyenne, barres par aspect, **piste d'amélioration** calculée sur l'aspect le plus faible) ;
- la répartition des réclamations par type de problème ;
- le droit de réponse public, avec un filtre « Sans réponse » ;
- l'accès au dossier d'une réclamation sur son produit pour répondre au client, et une notification à chaque nouvelle réclamation.



Avis sur mes produits : les aspects faibles apparaissent en rouge (fraîcheur des fraises, emballage des citrons et de la harissa).

13. Tests et validation

199	921	60	29/29
tests du projet tous au vert	assertions	tests du module (50 nouveaux)	pages HTTP par rôle

Fichier de tests	Ce qui est vérifié
`ReclamationTest` (20)	Création avec commande et lot, référence, échéance, priorité relevée ; validation ; commande d'un autre client refusée ; lot incohérent refusé ; modification interdite après prise en charge ; accès d'un autre client refusé ; producteur vendeur qui voit et répond ; notifications ; clôture et satisfaction unique ; relance d'un dossier en retard ; back office interdit aux clients ; copilote ; PDF ; analyse en arabe.
`AvisTest` (11)	Achat vérifié ; page produit (moyenne, synthèse) ; modération d'un avis injurieux invisible ; un avis par produit, pas sur son produit, pas par l'admin ; vote utile en bascule ; 3 signalements → modération ; droit de réponse refusé à un autre producteur ; avis négatif → réclamation (idempotent) ; modération admin.
`DeepSeekIATest` (10)	Aucun appel sans clé ; en-tête Bearer, modèle et mode JSON ; JSON entouré de Markdown ; valeur inventée écartée ; panne → règles locales + pause ; copilote ; responsable ; modération ; synthèse en cache (1 seul appel).
`AlerteQualiteTest` (8)	Seuil ; retards et questions exclus ; alerte avec lot et gravité ; pas de doublon ; analyse IA ; rappel (lot rappelé, clients notifiés, plus d'alerte ensuite) ; lot d'un autre produit refusé ; classement ; commande Artisan.
`AssistantSahbiTest` (11)	Accueil proactif ; accueil admin ; mode hors ligne ; boucle outil → réponse ; **carte de confirmation** (rien d'enregistré avant) ; annulation ; commande d'un autre client refusée au modèle ; outils admin absents pour un client ; carte d'un autre utilisateur interdite ; dossiers prioritaires pour l'admin ; réinitialisation.

Validation de bout en bout

- **Application lancée** (`php artisan serve`) et parcourue par HTTP rôle par rôle : 29 pages, codes attendus (200 / 403) respectés, aucune erreur PHP.
- **Formulaires réels** (CSRF) : publication d'avis, dépôt de réclamation, API de Sahbi, copilote et réanalyse : OK.
- **DeepSeek réel** avec la clé : analyse (derja), copilote (anglais, français, trois tons, 2 à 3 s), conversation Sahbi complète jusqu'à la création confirmée, triage admin, analyse d'alerte, modération, synthèse.
- Style : Laravel Pint appliqué aux fichiers du module.

> **Corrections apportées grâce aux tests réels :** Sahbi répétait des informations déjà données (ajout d'un rappel système juste avant la dernière demande) ; Sahbi demandait « tu confirmes ? » au lieu d'afficher la carte (règle explicite) ; icône absente du thème ; bulles de discussion qui affichaient des sauts de ligne parasites ; l'admin pouvait noter un produit (règle ajoutée dans la Policy).

14. Données de démonstration

`ReclamationSeeder` (lancé après les commandes, pour les achats vérifiés) crée un jeu de données qui couvre **tous les cas d'usage**, sans appel à l'IA (rapide et reproductible).

Élément	Contenu
Comptes	Comptes de l'équipe (`admin@` , `producteur@` , `client@` , `consommateur@` …) + 5 clients nommés (`amira@` , `youssef@` , `leila@` , `karim@` , `nour@nutritrace.tn`) + 12 clients pour le volume. Mot de passe : `password` .
16 réclamations	Tous les statuts ; 4 en retard ; délais respectés et dépassés ; satisfaction 5, 3 et 2 ; rejetée ; urgente sanitaire avec « qui est responsable » ; créée par Sahbi ; issue d'un avis ; en **arabe**, **derja** et **anglais** ; échanges à trois (admin, client, producteur) ; chronologies datées.
50 avis	8 produits, notes réalistes, aspects, achats vérifiés, réponses du producteur, votes utiles, 2 avis en modération (insulte + téléphone, lien), 1 masqué, signalements.
3 alertes qualité	**Ouverte** : fraises moisies (lot LOT-2026-0017, rappel conseillé). **Rappel déclenché** : plastique dans les piments (lot rappelé, clients notifiés). **Classée** : tomates trop mûres.
Notifications	Réponses aux clients et nouvelles réclamations pour le producteur.

15. Installation et scénario de démonstration

Installation

```
git checkout ReclamationReponse
composer install
cp .env.example .env          # puis renseigner DEEPSEEK_API_KEY=sk-...
php artisan key:generate
php artisan migrate:fresh --seed
php artisan serve             # http://127.0.0.1:8000
php artisan reclamations:alertes --ia   # (facultatif) détection + analyse IA des alertes
php artisan test              # 199 tests
```

À savoir :

- **Sous Windows**, si l'IA ne répond jamais (journal : « cURL error 60 »), PHP n'a pas de certificats racine : télécharger `cacert.pem` et renseigner `curl.cainfo` / `openssl.cafile` dans `php.ini`. Cela concerne aussi Gemini et OpenAI des autres modules.
- **Bug existant sur main (module Environnement)** : `EnvironnementSeeder` insère une colonne `source_ia` qui n'existe dans aucune migration, ce qui fait échouer `migrate:fresh --seed` avant les seeders suivants. Il est à corriger par le responsable du module (ajouter la colonne ou retirer la clé avant l'insertion).
- Sans clé DeepSeek, tout fonctionne avec les règles locales (badge « Mode simplifié » dans Sahbi).

Scénario de démonstration conseillé (≈ 8 minutes)

1. **client@nutritrace.tn** : ouvrir Sahbi → l'accueil propose de noter un produit reçu. Écrire « Les tomates de ma dernière commande étaient pourries » → Sahbi retrouve la commande → carte → *Confirmer* → la réclamation apparaît avec sa référence et son délai.
2. Écrire en derja : « chnowa sar fi réclamation mte3i ? » → statut réel et délai restant.
3. Fiche produit **Oranges maltaises** : synthèse IA, aspects, achat vérifié, réponse du producteur ; publier un avis 1★ qui décrit un problème → bandeau « Ouvrir une réclamation ».
4. **admin@** : tableau (indicateurs, retards en rouge) → dossier « Maux de ventre après les fraises » → *Analyser la chaîne* → copilote ton « Empathique » → envoyer.
5. **Alertes qualité** → analyser avec DeepSeek → rappeler le lot → ouvrir la page publique de traçabilité du lot : statut « Rappelé ».
6. **producteur@** : avis sur mes produits → pistes d'amélioration → répondre à un avis.
7. Sahbi côté admin : « Quels dossiers sont en retard ? » → liste triée avec liens.

16. Fichiers créés et modifiés

Back-end (créés)

app/Services/DeepSeekService.php
app/Services/AiReclamationService.php (réécrit)
app/Services/AvisIAService.php
app/Services/GestionReclamationService.php
app/Services/GestionAvisService.php
app/Services/AvisProduitService.php
app/Services/AlerteQualiteService.php
app/Services/Assistant/SahbiAgent.php
app/Services/Assistant/AssistantOutils.php
app/Models/Avis, AvisReaction, AlerteQualite, HistoriqueReclamation, AssistantConversation, AssistantMessage
app/Models/Reclamation.php (enrichi)
app/Policies/ReclamationPolicy, AvisPolicy
app/Notifications/ReclamationNotification, RappelProduitNotification
app/Http/Controllers/Front/AvisController
app/Http/Controllers/Back/AvisController, AlerteQualiteController
app/Http/Controllers/Acteur/RetourClientController
app/Http/Controllers/AssistantController, NotificationController
app/Http/Requests/StoreAvisRequest
app/Console/Commands/DetecterAlertesQualite
config/reclamations.php

Front, données, tests

resources/views/front/avis/* (section, champs, index, edit)
resources/views/front/reclamations/* (refondues)
resources/views/back/reclamations/* (+ pdf)
resources/views/back/avis/index, back/alertes/index
resources/views/acteur/retours/*
resources/views/partials/assistant/sahbi
resources/views/components/etoiles
public/css/reclamations-avis.css, sahbi.css
public/js/sahbi.js
database/migrations/2026_11_03_100000…100003
database/seeders/ReclamationSeeder (réécrit)
database/factories/AvisFactory
tests/Feature/ReclamationTest (réécrit)
tests/Feature/ReclamationsAvis/* (5 fichiers)

Fichiers partagés modifiés (ajouts ciblés)

routes/web.php, routes/console.php, config/services.php, .env.example, phpunit.xml, RolePermissionSeeder, DatabaseSeeder (ordre), CatalogueController::show (section avis), User / Product (relations), layouts front/back (widget), navbar, sidebar.

Commits sur **ReclamationReponse** : « module Réclamations & Avis avec IA DeepSeek et agent Sahbi », « données de démonstration complètes et finitions », « finitions issues des tests réels avec DeepSeek ».

17. Limites et pistes d'évolution

- **Pièces jointes** : ajouter des photos aux réclamations et aux avis (stockage Cloudinary déjà présent dans le projet), et analyser les photos avec un modèle de vision.
- **Temps réel** : notifications poussées (Laravel Reverb / WebSockets) au lieu d'un affichage au chargement.
- **Réponse en streaming** pour Sahbi (affichage mot à mot), et mémoire des préférences du client.
- **Statistiques avancées** : tableau de bord avec graphiques (évolution par catégorie, par producteur, par mois).
- **Coûts IA** : suivi du nombre d'appels et des jetons consommés, plafond par utilisateur.
- Les emails partent par la file d'attente : en production, lancer `php artisan queue:work` et le planificateur (`schedule:run`).

> **Conclusion.** Le module répond aux trois exigences du projet avec une approche cohérente. Les **rôles** sont appliqués par des Policies et des permissions, y compris pour l'agent. Les **fonctionnalités avancées** couvrent tout le cycle après-vente. L'**IA** apporte une vraie valeur à chaque étape (comprendre, prioriser, répondre, modérer, détecter, agir) tout en restant sous le contrôle de l'humain et en fonctionnant sans connexion. Il s'intègre aux modules de l'équipe (Commandes, Catalogue, Traçabilité) au lieu de les dupliquer.